Verifier Bias Is Not Enough: A Controlled Re-examination of Capability Collapse in Iterative Self-Training of Small Reasoning Models

Tan Sang Nguyen[1] and Duy Anh Tran[1]

Ton Duc Thang University, Ho Chi Minh City, Vietnam
52300057@student.tdtu.edu.vn, 52300006@student.tdtu.edu.vn

Abstract. Self-training loops (generate → verify → select → train, GVT) assume a reliable verifier. We tested whether a rule-based verifier with a directional format bias — `math-verify` rejects `\dfrac`/`\tfrac` rewrites of correct answers 100% of the time — causes capability collapse over rounds. A pooled GSM8K + MATH-500 run appeared to confirm it (Qwen2.5-1.5B pass@8 72.6% → 48.6%), but it shared one adapter across datasets and scored on the selection items. With isolated adapters, a held-out exam split and a matched patched-verifier control, the collapse disappears at our training rate (10^{-5}) and patching changes nothing across two paired seeds: the biased format is only ≈ 0.55% of generations, so impact is severity × prevalence. A late check qualifies this: 10^{-5} barely trains the adapter. At 10^{-4} (three seeds) accuracy falls by about two points, within noise, but the loss grows with the learning rate and at 5×10^{-4} all four runs lose accuracy by round 2. The patch's effect flips sign between seeds, and no-filter and train-on-gold controls (two seeds each) show the verifier is not the driver. The decline has a different anatomy in each regime: under self-training, generations that drift and never answer rise from about 10% to 40% at 5×10^{-4} while accuracy on finished solutions holds, and a larger token cap does not remove it, since the untrained model gains as much from it; under reference-solution training, responses halve in length and accuracy truly falls.

Keywords: self-training · verifier bias · mathematical reasoning · small language models · experimental confounds · pass@k.

1 Introduction

A common recipe for improving the mathematical reasoning of a small language model (SLM) *without human annotation* is to let the model generate its own solutions, keep those that an automated verifier judges correct, and fine-tune on the kept solutions, for several rounds. We call this the *Generate–Verify–Train* (GVT) loop; it underlies STaR [16], ReST [10], ReST-EM [9] and rejection-sampling fine-tuning [14]. Every such method assumes that the verifier separates correct from incorrect solutions reliably, which a *rule-based* verifier such as

`math-verify`[1] violates in one direction: it compares an extracted final answer with the gold answer after normalisation, so it almost never accepts a wrong answer but rejects correct answers written in forms it does not recognise. We measured this bias directly (Section 3.2): on GSM8K the false-negative rate on correct solutions is effectively zero, on MATH-500 it is about 7.8%, and some valid rewrites, such as `\dfrac` for `\frac`, are rejected 100% of the time.

This paper asks a single question: *when a directionally biased verifier is the filter in a multi-round GVT loop, does the model improve, merely learn the verifier's preferred style, or lose capability?* Our first experiments seemed to answer it decisively. Five rounds on a pooled GSM8K + MATH-500 set cut pass@1 and pass@8 sharply for two model sizes, and an earlier version of this work reported this as evidence that format-biased verifiers cause collapse. It does not survive a controlled re-examination. The pooled design trains *one* shared adapter on both datasets (GSM8K supplies about 69% of the mixture) and scores on the very items used for selection. Rebuilt with one adapter per dataset, a held-out 30% exam split and a matched patched-verifier control, the collapse disappears and the control is null, at the learning rate we used (10^{-5}). A late diagnostic then showed that rate to be too low to train the adapter at all, and the rest of the paper re-examines the result at rates where the model does learn.

Contributions.

- We characterise the directional bias of a real rule-based verifier by format probe and manual false-negative audit, and show that its impact is severity × prevalence: 100% rejection of a format that occurs in $\approx 0.55\%$ of generations discards half a percent of the training pool.
- We show that a pooled design with selection-set scoring manufactures a collapse that isolation and held-out evaluation do not reproduce, and that the matched patched-verifier control is null at two seeds.
- We show that this flatness is conditional on the learning rate: 10^{-5} barely trains the adapter, accuracy at 10^{-4} falls by about two points (within noise, three seeds), and the loss grows with the rate, reaching a decline at 5×10^{-4}. The patch's effect has no consistent sign across seeds, and no-filter and train-on-gold controls show the verifier is not the driver.
- We decompose the remaining decline and relate it to two recent studies (Section 2): under self-training it is mostly generations that drift and never answer, with accuracy on finished solutions unchanged; under reference-solution training it is shortened responses and a real loss of accuracy.

2 Related Work

Self-training and its limits. STaR [16] introduced the generate–filter–retrain loop, and ReST [10], ReST-EM [9] and RFT [14] formalise it; ReST-EM re-

[1] https://github.com/huggingface/math-verify, version 0.9.0 with antlr4 4.13.2 in this work.

ports diminishing test gains after the first round and an uninvestigated one-round regression on APPS. All treat the correctness signal as trustworthy. Yue et al. [15] show that RL raises pass@1 while the base model wins at large pass@k, and Teacher-Free Self-Training [11] observes a pass@8 versus pass@64 crossover on a free-verifier domain, concluding that self-training "amplifies but does not compound". Training repeatedly on model-generated data erodes distribution tails [8], and Qiao et al. [7] argue theoretically that a selector seeing only a narrow slice of the target distribution accelerates collapse.

Verifier error. TinyV [13] measures that rule-based verifiers reject many correct solutions and repairs them with an LLM check; From Accuracy to Robustness [5] reports about 14% false negatives for rule-based verifiers in mathematical RL, consistent with ours. Imperfect Verifiers [1] gives the most formal noise model, with fixed false-accept and false-reject probabilities, and notes that real noise can depend on formatting but then assumes content-independent rates in a single RL round. Egashira et al. [3] classify verifier errors in RLVR on a synthetic arithmetic task (Qwen3-1.7B-Base, OLMo3-7B) with hand-designed patterns such as format-based false negatives, and find that systematic false negatives behave like random noise — the model learns to avoid the triggering output and still improves — whereas false positives can cause plateau or collapse. Our control tests the same question with a *real* verifier whose bias we measured, real data, and several supervised rounds, and reaches the same conclusion.

Auditing self-improvement. Xu et al. [12] audit three rounds of rank-32 LoRA self-training of Qwen3-8B ($\eta = 10^{-4}$; filters by answer match, majority vote or an external teacher) against a frozen untrained control, and identify seven measurement failures that each invert a reported finding when its control is absent, including a fixed token cap under style drift and training-seed variance (the corruption-to-learning ratio of one method spans 0.55–1.53 across seeds). Self-training does not improve rarely-solved problems (0–2 of 22, against 8–11 for teacher distillation) and corrupts problems solved at baseline. This is the study closest to ours in spirit, and our results meet theirs at three points: the A-versus-B gap changes sign between seeds; a 150-problem exam set resolves only large effects; and self-training drifts in style, which costs the adapters answers (Section 4.6). They hold the filter and learning rate fixed and use no measured format-biased verifier; we vary the verifier and the learning rate and add reference-solution and no-filter controls, but have no frozen control or distillation arm.

Reference-guided training. Kim et al. [6] show that self-distillation conditioned on the ground-truth solution can cut mathematical accuracy by up to 40% (Qwen3-1.7B/8B, DeepSeek-Distill-Qwen-7B, Olmo3-7B-Instruct) while *shortening* responses, which they trace to suppressed epistemic verbalization. Our train-on-gold control is a supervised, small-model analogue with the same signature (response length −59%, accuracy on finished solutions 37% → 14%); we

did not measure epistemic markers, so we claim the signature, not the mechanism. Self-training shows the *opposite* length drift, which that literature does not predict.

Positioning. In the literature we surveyed (searched through October 2026) we found no study combining a real, measured, format-biased rule-based verifier with several sequential self-training rounds and a matched patched-verifier control. The closest are Egashira et al. (hand-designed errors, one RLVR phase) and Xu et al. (a measurement audit with answer, vote and teacher filters). We add that combination, an account of how pooling, selection-set scoring and an undertrained adapter each manufacture a misleading curve, and a decomposition of the decline that remains.

3 Setup

3.1 The GVT Loop

Each round t has four steps. *Generate:* the current model samples k solutions per problem at temperature 0.8. *Verify:* `math-verify` extracts the final answer and compares it with the gold. *Select:* accepted solutions are kept. *Train:* a *fresh* LoRA adapter (rank 16, $\alpha = 32$) is attached to the *base* model and trained by supervised fine-tuning on the kept solutions for one epoch; adapters are not accumulated, so round t learns only from round $t-1$'s accepted pool. Rounds are indexed from $t = 0$, the untrained base model. The first experiments use $k = 8$ and five rounds.

3.2 The Verifier and Its Directional Bias

Format probe. We re-rendered gold answers from GSM8K and MATH-500 in individually valid formats (bare number, trailing-zero decimal, `\frac`, `\dfrac`, `\tfrac`, boxed, dollar-delimited, sentence, with units) and asked the verifier to grade them against the original gold. On GSM8K plain rewrites are never rejected, while `\dfrac`/`\tfrac` are rejected 200/200 (the command is not parsed). On MATH-500, `\dfrac`/`\tfrac` are rejected 146/146 (100%), trailing-zero decimals 59%, and boxed/dollar-delimited forms about 20% of the time.

False-negative rate on real solutions. For genuinely correct public model solutions (teacher traces for GSM8K, Qwen2.5-7B rollouts for MATH-500) we checked manually, for each rejected solution, whether the boxed answer matched the gold. The false-negative rate is **7.8**% on MATH-500 (10/128) and **0**% on GSM8K (0/200), identical for the `default` and `reward` presets. Of the ten MATH-500 cases, eight are wrong-token extraction from a long solution and two are valid rewrites the verifier cannot match; we never saw it accept an incorrect final answer.

The patch. For the control we repaired the largest identified failure mode with a normalisation pass that rewrites `\dfrac` and `\tfrac` to `\frac` before parsing, removing the 100%-rejection class and nothing else.

Table 1. The two designs. Design I removes the pooling confound and the selection/evaluation overlap and adds the patched-verifier control.

	Design P (pooled)	Design I (isolated)
Datasets per run	GSM8K + MATH-500	one only
Adapter	one, shared	one per dataset
Training mixture	≈ 69% GSM8K	single-domain
Evaluation items	same as selection	held-out 30%
Verifier control	none	Run B (patched)
Models	0.5B and 1.5B	1.5B

3.3 Two Experimental Designs

Design P (pooled) loads both datasets together and trains *one* adapter per round on the union of accepted solutions, with selection and evaluation on the same problems. *Design I (isolated)* uses one dataset and one adapter per run, and holds out 30% of the problems (150 of 500) as an `exam` set never used for selection; all Design I pass rates are on the exam set. Table 1 summarises them.

Design I has three runs, all Qwen2.5-1.5B-Instruct, $n = 500$, five rounds, $k = 8$, differing in one respect each: **Run A** (MATH-500, original verifier), **Run B** (MATH-500, patched verifier; the single-variable control) and **Run C** (GSM8K, original verifier, already near-clean). Runs A and B were repeated at a second seed; since the seed also sets the train/exam partition, the repetition draws a different 150-problem exam set.

3.4 Models, Data, Training, Metrics

We use Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct (Design P uses both, Design I the 1.5B model only) and 500 problems each from GSM8K [2] and MATH-500, the 500-problem subset of MATH [4]. Training is LoRA supervised fine-tuning (`transformers`, `peft`, TRL `SFTTrainer`), one epoch per round, at a learning rate of 10^{-5} unless stated otherwise;[2] we use SFT rather than online RL because it matches the select-then-train structure of the loop. Metrics per round are pass@1 and pass@k, style counts over all generations (boxed, dfrac, dollar, ...), pre-filter answer diversity (distinct normalised answers among the k samples), and the training loss on the kept pool. All runs use one Kaggle T4 GPU; Design P and Run C were run once, Runs A and B at two seeds.

4 Results

4.1 Design P: An Apparent Collapse

Table 2 reports pass@1 and pass@8 for both models over five rounds of the pooled design (the two verifier presets give identical values). All series decline,

[2] A post-hoc diagnostic shows this rate undertrains the adapter (Section 4.5 and Limitation 1).

Table 2. Design P: pass@1 and pass@8 over five GVT rounds, pooled GSM8K + MATH-500, scored on the selection set. Round 0 is the untrained base model. Section 4.1 explains why these numbers cannot be attributed to verifier bias.

Model	Metric	R0	R1	R2	R3	R4	Δ (R0→R4)
0.5B	pass@1	31.1%	31.9%	29.9%	28.9%	27.4%	−3.7 pt
0.5B	pass@8	60.5%	58.9%	56.1%	55.5%	54.9%	−5.6 pt
1.5B	pass@1	50.0%	46.2%	44.5%	32.4%	18.6%	**−31.4** pt
1.5B	pass@8	72.6%	71.5%	70.2%	63.6%	48.6%	**−24.0** pt

Table 3. Design P, 1.5B, split by dataset. GSM8K has a measured 0% verifier false-negative rate yet declines at least as much as MATH-500.

Dataset	Metric	R0	R1	R2	R3	R4	Δ
GSM8K	pass@1	69.6%	62.0%	61.0%	45.6%	26.6%	−43.0 pt
GSM8K	pass@8	91.4%	90.4%	89.0%	82.2%	65.8%	−25.6 pt
MATH-500	pass@1	30.4%	30.4%	28.0%	19.2%	10.6%	−19.8 pt
MATH-500	pass@8	53.8%	52.6%	51.4%	45.0%	31.4%	−22.4 pt

and the 1.5B decline is severe and accelerating. Splitting the 1.5B run by dataset (Table 3) sharpens the puzzle: under the original hypothesis MATH-500 (false-negative rate 7.8%) should degrade and GSM8K (0%) should hold, but both fall heavily and GSM8K loses more pass@1 in points. The accepted pool also contracts every round (0.5B −16%, 1.5B −60%, from 3,962 to 1,566 solutions) and 1.5B output style shifts, with `\boxed{}` falling from 5,608 to 2,693 occurrences while dollar-delimited math rises from 1,453 to 3,406. Read alone, this is a coherent story of a shrinking, style-skewed pool driving capability down.

Two properties of Design P block that inference. First, each round trains one adapter on the union of both datasets' accepted solutions, of which GSM8K supplies 68.4–70.2% in every round; either dataset's trajectory is therefore a joint function of both pools, and no contrast within the design isolates the verifier. Second, it scores each round on the problems it also used to select training data, so a round-over-round change mixes capability change with fit to those items.

4.2 Design I: The Collapse Does Not Reproduce at the Original Learning Rate

Table 4 reports the isolated runs on their held-out exam sets. The collapse is absent: pass@8 is flat in every run — 57.3% → 56.0% and 58.7% → 58.7% for Run A at seeds 0 and 1, 57.3% → 53.3% for Run B, 92.7% → 90.0% for Run C — against declines of 22–26 points in the pooled design. On MATH-500 pass@1 falls about three points at round 1 and is flat thereafter, and the two seeds agree (−2.7 and −3.4 points end to end); rounds 1–4 show no trend in either direction. This is a one-off cost of the first self-training round rather than a compounding process, which fits the "amplifies but does not compound" reading of [11]. Run C

Table 4. Design I: pass@1 and pass@8 on the held-out exam set (150 problems), Qwen2.5-1.5B. Runs A and B differ only in whether the verifier's format bias is patched. Run A is shown for both seeds; seed 1 draws a different exam partition, so its round-0 value differs (Section 4.2).

Run	Metric	R0	R1	R2	R3	R4	Δ
A, seed 0 (MATH-500, orig.)	pass@1	32.0%	29.3%	28.0%	29.3%	29.3%	−2.7 pt
A, seed 0 (MATH-500, orig.)	pass@8	57.3%	56.7%	54.7%	57.3%	56.0%	−1.3 pt
A, seed 1 (MATH-500, orig.)	pass@1	36.7%	32.7%	32.7%	31.3%	33.3%	−3.4 pt
A, seed 1 (MATH-500, orig.)	pass@8	58.7%	60.7%	59.3%	59.3%	58.7%	0.0 pt
B, seed 0 (MATH-500, patched)	pass@1	32.0%	28.7%	28.7%	28.7%	28.7%	−3.3 pt
B, seed 0 (MATH-500, patched)	pass@8	57.3%	55.3%	52.0%	57.3%	53.3%	−4.0 pt
B, seed 1 (MATH-500, patched)	pass@1	37.3%	32.0%	32.7%	31.3%	33.3%	−4.0 pt
B, seed 1 (MATH-500, patched)	pass@8	59.3%	60.7%	58.0%	58.7%	58.7%	−0.6 pt
C, seed 0 (GSM8K, orig.)	pass@1	66.7%	66.7%	65.3%	63.3%	54.0%	−12.7 pt
C, seed 0 (GSM8K, orig.)	pass@8	92.7%	90.7%	93.3%	90.0%	90.0%	−2.7 pt

retains a residual effect: GSM8K pass@1 falls 12.7 points, almost all in the final round (63.3% → 54.0%), while its pass@8 does not move; with 150 problems and one seed we flag it as an open question, not a finding (Section 4.5 reruns it at a higher learning rate). The accepted pools contract here as well (Run C −20%, Run A −3%) without any pass@8 decline, so pool contraction and collapse, which moved together in Design P, come apart.

Noise floor. Round 0 is the untrained base model, identical across seeds; only the exam partition differs, yet round-0 pass@1 is 32.0% at seed 0 and 36.7% at seed 1 (4.7 points, seven problems; pass@8 differs by 1.4). This is a between-partition variance. Most comparisons here are paired within a partition (round 0 against round 4 in one run, A against B at one seed), where it largely cancels, but it bounds comparisons of absolute levels across seeds, shows that a 150-problem exam set resolves differences of a few points only weakly — the regime of the A-versus-B gap of 0.6 points — and puts Run C's 12.7-point decline outside what partition choice explains.

4.3 The Control: Patching the Verifier, and Why It Changes Nothing

Runs A and B are identical in dataset, seed, split, model and training configuration and differ only in whether `\dfrac`/`\tfrac` are normalised before verification — the one error class measured at 100% rejection. The curves are indistinguishable (Table 4): after five rounds pass@1 ends at 29.3% unpatched against 28.7% patched (one problem of 150) and pass@8 at 56.0% against 53.3%, with Run B oscillating non-monotonically (55.3 → 52.0 → 57.3 → 53.3); answer diversity and per-round training loss are near-identical. At seed 1 the agreement is closer still: from round 2 the two runs report identical pass@1 (32.7%, 31.3%, 33.3%),

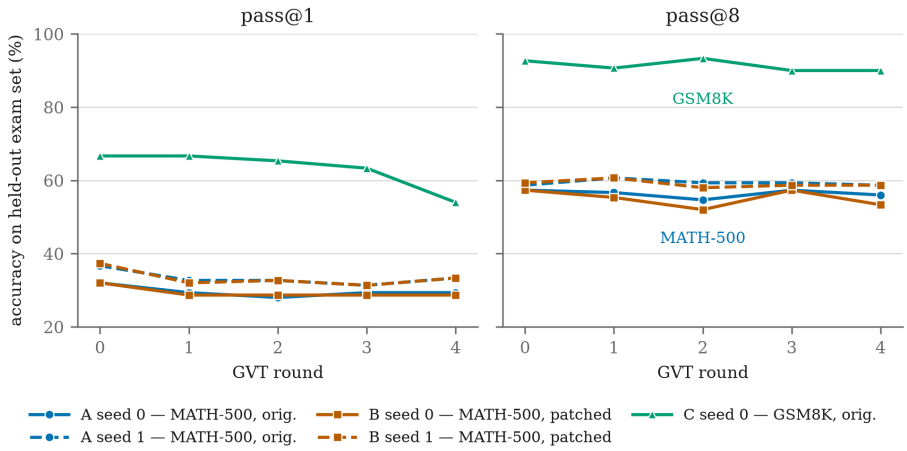


Fig. 1. Design I: pass@1 (left) and pass@8 (right) on the held-out exam set, by round. Colour identifies the run, line style the seed, marker the run again, so the figure survives greyscale printing. Runs A and B — identical except for the verifier patch — track each other closely at both seeds. pass@8 is flat in every run, in contrast to the 22–26 point declines of Design P. The one visible movement is Run C's pass@1 in the final round.

and across both seeds no round differs by more than 2.7 points, with no consistent sign. Within the precision of this experiment the hypothesis that directional format bias drives multi-round degradation is *not supported* at this learning rate.

The reason is prevalence. At round 0, Runs A and B generate identical solutions from the same base model, so the patch's direct scoring effect is isolated: zero problems of 150 at seed 0 and one at seed 1. Across the 4,000 generations per round on MATH-500, `\dfrac` appears in 20–29 (about 0.55%), `\tfrac` never, and on GSM8K neither occurs. The format probe's 100% is severity *conditional on the format occurring*; what governs the training pool is severity × prevalence, here 0.55% of solutions wrongly discarded per round, which cannot plausibly drive a twenty-point change. Our audit agrees: of ten MATH-500 false negatives only two were format rewrites, and eight were wrong-token extraction that the patch does not touch. Verifier-bias figures in the literature, such as the ∼14% of [5] and the rejected fractions of [13], are conditional severities measured on some reference distribution; whether they matter depends on how often the affected forms occur in *that pipeline's* output, which is cheap to count and which we did not count until the hypothesis had failed.

4.4 Pre-Filter Answer Diversity Rises, It Does Not Collapse

Does the model converge onto one *answer* per problem before the verifier sees the samples? We measured the mean fraction of distinct normalised answers among the $k = 8$ samples and the fraction of problems on which all k agree.

Table 5. Runs A and B rerun at 5×10^{-4} (MATH-500, $k = 4$, three rounds), held-out exam set of 150 problems. B differs from A only in the verifier patch; each seed draws its own exam partition. "Boxed" is the share of the 2,000 generations in each round containing `\boxed`.

Run, seed	Round	pass@1	pass@4	Kept pool	Boxed
A (orig.), 1	0	33.3%	50.0%	440	52%
	1	32.7%	49.3%	391	51%
	2	23.3%	42.7%	326	36%
B (patched), 1	0	33.3%	50.7%	443	52%
	1	35.3%	50.0%	392	49%
	2	28.7%	48.7%	328	41%
A (orig.), 2	0	31.3%	51.3%	424	54%
	1	31.3%	46.0%	386	50%
	2	27.3%	44.7%	350	37%
B (patched), 2	0	31.3%	52.0%	424	54%
	1	31.3%	44.7%	398	50%
	2	20.0%	40.7%	332	38%

Diversity rises in both designs, the opposite of a narrowing picture, and its size tracks the pass@1 decline: all-k agreement falls 24.8% $\rightarrow$ 1.7% for Design P's 1.5B run, 10.5% $\rightarrow$ 7.3% for 0.5B and 33.6% $\rightarrow$ 16.0% for Run C, but only 14.6% $\rightarrow$ 12.0% (A) and 14.6% $\rightarrow$ 12.2% (B) at seed 0, whose pass@1 barely moves. This fits a single underlying quantity, first-attempt reliability, rather than stylistic convergence: a model that still solves a problem tends to land on the same answer across attempts, while one whose reliability erodes answers more erratically, which depresses pass@1 while leaving pass@8 comparatively intact.

4.5 A Learning-Rate Check: The Flatness Is Conditional

Every run above used a LoRA learning rate of 10^{-5}. A one-round diagnostic on the same MATH-500 pool (Limitation 1) shows that rate barely trains the adapter: training loss oscillates between 0.68 and 0.84 with no downward trend, against a fall to 0.24–0.28 at 5×10^{-4}. The flat curves and the null control could therefore reflect a training step that hardly moves the model. We reran Run A and its control Run B at 5×10^{-4} at two seeds, and Run A at 10^{-4} at three seeds and at 2×10^{-4} at two, changing only the budget (three rounds and $k = 4$, so pass@4 replaces pass@8). Seed 1 shares the exam partition of the seed-1 runs above and seed 2 draws a new one; round-0 values differ from the earlier runs' only through sampling. The loss falls within round 0 from about 0.5 to 0.25 at 5×10^{-4} and from about 0.6 to 0.28 at 10^{-4}.

Table 5 shows three things. *At* 5×10^{-4} *the decline replicates:* by round 2 all four runs have lost accuracy, pass@1 by 10.0 and 4.0 (A) and 4.6 and 11.3 (B) points and pass@4 by 7.3, 6.7, 2.0 and 11.3 points, where one problem is 0.67 points; at the original rate the same seed-1 split's pass@1 moved -4.0 at

Table 6. Run A (MATH-500, original verifier, $k = 4$, three rounds) across learning rates and seeds: pass@1, pass@4 and sample accuracy (600 exam generations) at round 0 $\rightarrow$ round 2, and the degenerate share of round-2 generations. The two 5×10^{-4} rows are the A rows of Table 5.

Rate	Seed	pass@1	pass@4	Sample acc.	Degenerate (R2)
10^{-4}	1	33.3 $\rightarrow$ 34.0	50.0 $\rightarrow$ 49.3	33.7 $\rightarrow$ 32.2	19.5%
10^{-4}	2	31.3 $\rightarrow$ 28.0	51.3 $\rightarrow$ 45.3	30.8 $\rightarrow$ 27.2	19.5%
10^{-4}	3	29.3 $\rightarrow$ 26.7	40.0 $\rightarrow$ 41.3	28.5 $\rightarrow$ 27.5	22.7%
2×10^{-4}	1	33.3 $\rightarrow$ 31.3	50.0 $\rightarrow$ 46.7	33.7 $\rightarrow$ 30.2	25.0%
2×10^{-4}	2	31.3 $\rightarrow$ 25.3	51.3 $\rightarrow$ 45.3	30.8 $\rightarrow$ 27.0	22.2%
5×10^{-4}	1	33.3 $\rightarrow$ 23.3	50.0 $\rightarrow$ 42.7	33.7 $\rightarrow$ 25.3	41.3%
5×10^{-4}	2	31.3 $\rightarrow$ 27.3	51.3 $\rightarrow$ 44.7	30.8 $\rightarrow$ 26.2	36.3%

round 1 and then stayed within 1.4 points, and pass@8 did not move. Two of the four pass@1 declines and three of the four pass@4 declines exceed the 4.7-point partition noise; the others are individually within it, so the evidence is the agreement in direction across all four runs. *The loss of accuracy grows with the rate* (Table 6): at 10^{-4} the adapter trains and, over three seeds, sample accuracy at round 2 falls by 2.0 points on average (-1.5, -3.6, -1.0) and pass@1 and pass@4 by 1.8 each, individually within the 4-point noise of 150 problems apart from one pass@4 change of -6.0; at 2×10^{-4} by 3.7 (-3.5, -3.8) and at 5×10^{-4} by 6.5 (-8.4, -4.6). *The patch has no consistent effect:* at seed 1 Run B sits 5.4 points of pass@1 and 6.0 of pass@4 above Run A at round 2, at seed 2 it sits 7.3 and 4.0 points *below*. The sign flips with the seed, so the gap is trajectory noise; it exceeds every end-to-end A-versus-B difference at 10^{-5} (at most 2.7 points), which means that where the adapter moves, runs of nearly the same configuration end several points apart and the noise floor measured at 10^{-5} is too small for this regime. The patch changes the first training pool by 0–3 problems, the prevalence account of Section 4.3 once more.

Two controls at 5×10^{-4}. The *no-filter* run trains on all k samples of every selection problem regardless of correctness (1,400 examples, about two thirds of which the verifier would reject), removing the verifier from the loop. The *train-on-gold* run ignores the model's generations and trains on the 350 reference solutions, identical every round, so its rounds are replicate fine-tunes of the base model and the difference between its rounds 1 and 2 measures fit-to-fit spread. Both were run at two seeds, on the partitions of Run A at the same seeds.

Table 7 gives two readings, and both replicate across seeds. Removing the filter does not help: the no-filter run ends at 27.3% and 40.7% (seed 1) and 22.7% and 39.3% (seed 2), against Run A's 23.3% and 42.7% and 27.3% and 44.7%, differences within the seed-to-seed spread and of no consistent sign, even though the verifier gate keeps only about 31% of the samples. Training on reference solutions is *worse* than self-training: one pass takes pass@1 from 33.3% to 18.0% and pass@4 from 50.0% to 33.3% at seed 1 (its replicate round gives 15.3% and

Table 7. Controls at 5×10^{-4} (MATH-500, $k = 4$, three rounds), on the exam partition of Run A at the same seed. Pool is the number of training examples per round.

Run	Pool	pass@1			pass@4		
		R0	R1	R2	R0	R1	R2
Seed 1							
A (verifier-filtered)	440→326	33.3	32.7	23.3	50.0	49.3	42.7
No filter	1400	33.3	26.7	27.3	50.0	49.3	40.7
Train on gold	350	33.3	18.0	15.3	50.0	33.3	29.3
Seed 2							
A (verifier-filtered)	424→350	31.3	31.3	27.3	51.3	46.0	44.7
No filter	1400	31.3	34.7	22.7	51.3	48.7	39.3
Train on gold	350	31.3	8.7	10.0	51.3	28.7	30.7

29.3%) and from 31.3% to 8.7% and from 51.3% to 28.7% at seed 2, while the model adopts the reference style (the share of generations with a `$`-delimited span goes from 5% to 98% at seed 1). The decline at this rate therefore does not depend on the verifier or on the correctness of the training data; self-generated data is the gentler source, plausibly because it lies closer to the model's own distribution, which we have not tested. Section 4.6 shows that the two declines have different anatomies. No run in this study showed a sustained pass@k improvement under self-training, which fits the "amplifies but does not compound" reading [11], and none is a collapse in the sense of Design P (pass@4 falls 2–11 points here, not 22–26).

Run C at the corrected rate. We also reran Run C (GSM8K, original verifier, seed 1, three rounds, $k = 4$) at 5×10^{-4}. It declines faster than any MATH-500 run: pass@1 falls 70.7% → 64.7% → 48.7% and pass@4 86.0% → 86.0% → 76.7%, against a pass@1 of 65.3% at round 2 at 10^{-5}. The degenerate share rises from 2.5% to 16.7% and mean length from 958 to 1,313 characters, and, unlike on MATH-500, accuracy on the finished generations also falls (71.5% → 64.6%), so on GSM8K the loss is not only a termination effect. The residual first-attempt erosion of Section 4.2 is therefore learning-rate dependent and larger at 5×10^{-4}.

4.6 Anatomy of the Decline

Sample-level accuracy mixes two things: whether a model that finishes a solution gets it right, and whether it finishes at all. We separate them on the exam generations of every corrected-rate run. A generation is *degenerate* if it is longer than 1,500 characters and contains no `\boxed`; this is a heuristic proxy for a response that never reached an answer within the 512-token generation cap, and its accuracy is 1.7–7.8% in every run and round.

Table 8 shows two different failures. In all six self-training runs at 5×10^{-4} the degenerate share rises from 9–10% to 36–43% and mean length grows by

Table 8. What the round-2 decline consists of (MATH-500, corrected learning rate, exam generations, round 0 → round 2). "Degenerate" is the share of generations longer than 1,500 characters with no `\boxed`; "non-degenerate accuracy" is sample accuracy on the remaining generations. "Truncated" is the share of the round-0 training pool longer than the 512-token training limit. The first six rows are at 5×10^{-4}, then 2×10^{-4} and 10^{-4}; the 10^{-5} row is the original run ($k = 8$, so its pool is not comparable); the last two rows (train on gold) are at 5×10^{-4}.

Run	Seed	Length (chars)	Degenerate	Non-degen. acc.	Truncated
A (orig.)	1	1175 → 1678	9.8 → 41.3%	37.2 → 38.9%	24.5%
B (patched)	1	1175 → 1659	9.8 → 35.7%	37.2 → 40.2%	24.6%
A (orig.)	2	1154 → 1641	8.7 → 36.3%	33.2 → 36.6%	24.5%
B (patched)	2	1154 → 1728	8.7 → 41.7%	33.6 → 34.9%	24.5%
No filter	1	1175 → 1668	9.8 → 38.8%	37.2 → 38.4%	62.4%
No filter	2	1154 → 1755	8.7 → 42.7%	33.2 → 34.0%	—
A, 2×10^{-4}	1	1175 → 1433	9.8 → 25.0%	37.2 → 38.9%	24.5%
A, 2×10^{-4}	2	1154 → 1431	8.7 → 22.2%	33.2 → 32.5%	24.5%
A, 10^{-4}	1	1175 → 1276	9.8 → 19.5%	37.2 → 38.7%	24.5%
A, 10^{-4}	2	1154 → 1284	8.7 → 19.5%	33.2 → 32.5%	24.5%
A, 10^{-4}	3	1178 → 1355	9.3 → 22.7%	31.2 → 34.5%	—
A, 10^{-5}	1	1158 → 1146	8.8 → 8.8%	38.5 → 37.2%	—
Train on gold	1	1175 → 485	9.8 → 0.0%	37.2 → 14.0%	17.4%
Train on gold	2	1154 → 516	8.7 → 0.3%	33.2 → 12.2%	—

40–50%, while accuracy on the non-degenerate generations does *not* fall (33–37% → 34–40%). Holding the degenerate share at its round-0 level, Run A at seed 1 would score 35.7% at round 2, above its round-0 33.7%. The round-2 decline is therefore mainly generations that do not finish, not a loss of accuracy on those that do (the conditioning is descriptive: if harder problems are the ones that degenerate, accuracy on the rest would rise without any change in ability). The degenerate generations are not long legitimate solutions cut by the cap: they open as ordinary solutions and drift into free-association text ("... acknowledged hurdles acknowledged setbacks acknowledged failures ..."), with low exact repetition (mean 5-gram repeat ratio 0.02, the same as finished generations), so they are not loops. Train-on-gold is the opposite: responses *shorten* by 59%, the degenerate share vanishes, and accuracy on finished generations falls from 37% to 14% — a real loss of accuracy, in the direction Kim et al. [6] report. Across learning rates the drift is a dose–response (Table 6): the degenerate share at round 2 is 8.8% at 10^{-5}, 19.5–22.7% at 10^{-4} (three seeds), 22.2–25.0% at 2×10^{-4} and 36.3–41.3% at 5×10^{-4}, while sample accuracy falls by 0.9 (at 10^{-5}, $k = 8$), 2.0, 3.7 and 6.5 points.

Two tests of the pipeline. The training limit (`max_length`= 512 tokens) equals the generation cap, so 24–26% of the kept pool (108 of 440 examples in Run A at round 0) is truncated: it loses its final answer and end-of-sequence token and plausibly teaches the model not to stop. We reran Run A at seed 1 and 5×10^{-4}

Table 9. Adapters re-scored on the same exam problems (seed-1 partition, $k = 4$, 150 problems) under a 512- and a 1,536-token generation cap. Round-2 adapters from runs at 5×10^{-4} ("replicate" keeps the 512-token training limit, "1024 limit" trains with no truncated example) and 2×10^{-4}, against the untrained model. pass@1 and pass@4 are noisy on 150 problems (re-scoring the same adapter moves them by about 4 points); sample accuracy and the degenerate share rest on 600 samples.

Model	Cap	pass@1	pass@4	Sample acc.	Degenerate	Acc. finished
Untrained	512	36.0%	52.0%	36.3%	8.7%	39.2%
	1536	40.7%	59.3%	43.2%	2.5%	44.1%
5×10^{-4}, replicate	512	29.3%	46.7%	28.3%	30.0%	38.1%
	1536	35.3%	52.0%	34.2%	23.0%	43.3%
5×10^{-4}, 1024 limit	512	26.0%	45.3%	27.5%	37.2%	40.8%
	1536	30.7%	50.7%	29.5%	32.7%	41.3%
2×10^{-4}	512	27.3%	48.7%	28.5%	28.7%	37.9%
	1536	32.0%	52.7%	32.0%	23.7%	40.8%

with a training limit of 1,024 tokens, so that no kept example is truncated, and re-scored the round-2 adapter of that run and of a matched replicate that keeps the 512-token limit on the same exam problems ($k = 4$). The drift does not fall: at the 512-token generation cap the degenerate share is 37.2% with the longer limit against 30.0% in the replicate (600 samples each, standard error about 2 points). Truncated training examples are therefore not what makes the models drift, which also fits the no-filter run, whose pool is 62–68% truncated and whose degenerate share (38.8%) is no higher than that of the filtered runs. The replicate also reproduces the decline of Run A at seed 1 (pass@1 33.3 → 31.3 → 25.3%, pass@4 50.0 → 48.0 → 44.0%, against 23.3% and 42.7% at round 2 in the first run).

Re-scoring the same adapters under a 1,536-token cap, with everything else fixed (Table 9), shows that the cap is not the explanation. The untrained model gains as much from the larger cap as the adapters do (sample accuracy 36.3% → 43.2%, degenerate share 8.7% → 2.5%), so the deficit of the round-2 adapters against it does not close: −8.0, −8.8 and −7.8 points at 512 tokens for the replicate, the 1024-limit and the 2×10^{-4} adapter, and −9.0, −13.7 and −11.2 at 1,536. Accuracy on finished generations matches the untrained model's at both caps (38.1% against 39.2% at 512 and 43.3% against 44.1% at 1,536 for the replicate), but the untrained model finishes 97.5% of its generations at 1,536 tokens and the adapters 67–77%. The adapters thus have a lasting termination problem that more tokens do not remove, and it shrinks with the learning rate: re-scored on their own exam sets, the 10^{-4} adapters of seeds 2 and 3 leave 6.2% and 12.8% of generations unfinished at 1,536 tokens. The fixed cap of Xu et al. is therefore not the whole story here: the style change is real, and it costs the adapters answers whatever the cap.

5 Discussion

5.1 Interpretation

An earlier version of this work explained the Design P curves with a self-reinforcing shrinking-pool mechanism: the biased verifier accepts a smaller, more style-skewed set each round and the skew compounds. It is coherent and matches Qiao et al. [7] theoretically, but the isolated runs do not support it. The pool contracts under Design I as well (20% in Run C) with no pass@8 decline, and removing the bias that supposedly drives the skew changes nothing (Section 4.3). What remains is narrower: in the pooled design each dataset's update was dominated by the other's filtered data and evaluation ran on the selection items, which suffices to produce the observed curves without any appeal to verifier bias.

Attribution also needs a caveat on our own comparison. Design I differs from Design P in three respects at once (isolation, a held-out split, and n per run falling from 1,000 pooled problems to 500), so we cannot apportion the difference among them and do not claim that isolation alone removes the collapse. The clean comparisons are internal to Design I and to its corrected-rate counterparts, where runs share every configuration except the one variable varied; the verifier control rests on those, not on the P/I contrast. That control agrees with Egashira et al. [3], who find that systematic false negatives in one-round RLVR behave like random noise: two quite different setups converging on "systematic false negatives are tolerable" is the most useful positive statement this study can make.

5.2 Practical Implications

For GVT-style pipelines our experience suggests six precautions. *Do not pool datasets into one adapter* if you report per-dataset trajectories: a mixture that is 69% one domain gives per-domain results that cannot be read independently, and isolation costs less than it appears. *Hold out evaluation items from selection*, the single change most likely to manufacture a dramatic round-over-round curve. *Verify that the training step fits:* a flat curve from an adapter whose loss does not fall is not evidence of stability, and a one-round diagnostic would have shown it before any multi-round run. *Report length and termination, not only accuracy:* the accuracy of finished solutions did not fall while the share of unfinished ones quadrupled, a style-drift failure of the kind Xu et al. [12] document. *Run the no-filter, train-on-gold and null controls:* a short SFT pass degraded our model whatever the data were, and repairing the suspected defect and re-running is the cheapest decisive experiment — our central claim survived two model sizes, tens of thousands of generations and a coherent mechanism story, and failed the first time a single-variable control was run against it. *Track pass@k alongside pass@1:* a rising pass@1 is not sufficient evidence of improvement, and Run C shows the converse, a falling pass@1 with flat pass@8 that would be misread as capability loss if pass@1 were the only metric.

6 Limitations

1. *The headline null result was obtained at a learning rate that undertrains.* Every run in Sections 4.2–4.3 uses 10^{-5} (Section 3.4). A one-round diagnostic on the same MATH-500 pool, every other hyperparameter fixed, shows training loss oscillating between 0.68 and 0.84 with no clear downward trend at 10^{-5} (six logged steps: 0.836, 0.729, 0.678, 0.737, 0.699, 0.724), against a drop to the 0.24–0.28 range at 5×10^{-4} (0.626, 0.354, 0.242, 0.240, 0.248, 0.281). The flat curves and the null control of Design I were therefore obtained with an adapter that barely moved; Section 4.5 reruns the loop at higher rates, and the Design P/Design I contrast itself was measured at 10^{-5} only.
2. *The corrected-rate evidence is thin.* Every corrected-rate run has three rounds, $k = 4$ and a 150-problem exam set. A and B have two seeds at 5×10^{-4}, A has three at 10^{-4} and two at 2×10^{-4}, and the no-filter and train-on-gold controls have two; Run C at the corrected rate and the token-cap re-scoring are single seeds, and the learning-rate grid is coarse. The patched-versus-unpatched gap flips sign between seeds, so we find no consistent patch effect, but cannot rule out a small one.
3. *The anatomy of the decline rests on a heuristic.* A generation is called degenerate if it is longer than 1,500 characters and has no `\boxed`; accuracy on the remaining generations is descriptive, and conditioning on it can raise accuracy without any change in ability if harder problems are the ones that degenerate.
4. *Limited seed replication at the original rate.* Runs A and B were run at two seeds, which agree; Run C and the pooled runs were run once, and Run C's 12.7-point pass@1 decline is the one effect at 10^{-5} that exceeds the noise floor.
5. *Small exam set.* Design I scores on 150 held-out problems, where one problem is 0.67 points and re-partitioning alone moves round-0 pass@1 by 4.7 points (Section 4.2); differences of a few points, including the entire A-versus-B gap, are below the resolution of this setup.
6. *Three variables change between designs* (Section 5.1), so the Design P/Design I contrast is suggestive rather than attributive, and Design I covers one model size, so we cannot say whether the apparent "larger collapses harder" pattern of Design P also dissolves under isolation.
7. *One patch, one bias class, one verifier, one training method.* Run B repairs `\dfrac`/`\tfrac` parsing, the error class measured at 100% rejection, not the eight-of-ten false negatives caused by wrong-token extraction, so the null concerns format-class bias specifically. Whether results change beyond five rounds, under another verifier, or under full fine-tuning or online RL is untested.

7 Conclusion

We asked whether a directionally biased rule-based verifier causes capability collapse when used as the filter in a multi-round GVT loop. Pooled experiments scored on the selection items produced a dramatic collapse, but isolating each dataset, holding out an exam split and adding a matched patched-verifier control removes it at our training rate, and the control is null at two seeds. The reason is prevalence: the affected format is about half a percent of generations, so a 100% rejection rate discards half a percent of the training pool.

That rate was too low to train the adapter. At 10^{-4} the adapter trains and accuracy falls by about two points over three seeds, within noise; at 5×10^{-4} all four runs, patched and unpatched, lose accuracy by round 2, and the loss grows with the learning rate. The verifier control survives in a weaker form: the patch's effect flips sign between seeds, the decline occurs with no filter at all, and training on reference solutions is worse, at both seeds of each control; on GSM8K the decline at 5×10^{-4} is larger still. What the decline is made of differs by regime (Section 4.6): under self-training it is mostly generations that drift and never answer, with accuracy on finished solutions unchanged, not caused by truncated training examples, and not removed by tripling the generation cap (the untrained model gains as much from it); under reference-solution training it is shortened responses and a real loss of accuracy. Read as accuracy under a fixed token cap, the first looks like lost reasoning, the measurement trap Xu et al. warn of, although here the unfinished answers are lost at any cap.

The finding we can defend is therefore methodological rather than mechanistic. Pooling and selection-set scoring together suffice to produce a collapse signature that isolation and held-out evaluation do not reproduce; an undertrained adapter produces reassuringly flat curves; and accuracy alone hides whether a model finishes its answers. Each was found only because a control was run against it, and we report the reversals in full because the designs that produced them are not unusual.

Future work. Characterise the generations that still never answer under a large cap (they open normally and drift into unrelated text) and test whether decoding settings or an end-of-sequence penalty remove them; refine the learning-rate grid between 10^{-4} and 5×10^{-4} and replicate Run C (GSM8K) at further seeds and rates; extend the corrected-rate runs to five rounds with $k = 8$ so they compare directly with Table 4; and extend the isolated design to a second model size.

A second format-class defect gives a natural next control. `math-verify`'s rich LaTeX extraction only activates inside a recognised delimiter (`\boxed{...}` or `$...$`), while our gold answers are stored undelimited, so a correctly boxed compound answer can be compared against a truncated gold stub. The format probe attributes 17/41 of MATH-500 `boxed` rejections to this mechanism (8.5%; the other 24/41 are pre-existing unparseable gold formats), an estimated damage of $8.5\% \times 51.5\% \approx 4.4\%$ of solutions under the measured prevalence — roughly eight times `\dfrac`'s 0.55% and untested by any study we know of. We implemented and unit-tested a patch (wrap gold in `\boxed{}` whenever the prediction

contains one and gold does not) and queue it as a Run-D control against the headline configuration, as Run B was against Run A.

Acknowledgments. This work was carried out as an undergraduate research project. No external funding was received.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Cai, X.-Q., Wang, W., Liu, F., Liu, T., Niu, G., Sugiyama, M.: Reinforcement Learning with Verifiable yet Noisy Rewards under Imperfect Verifiers. arXiv:2510.00915 (2025)
2. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 (2021)
3. Egashira, K., Vero, M., Dekoninck, J., Dörner, F.E., Staab, R., Vechev, M.: Delay, Plateau, or Collapse: Evaluating the Impact of Systematic Verification Error on RLVR. arXiv:2605.02909 (2026)
4. Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J.: Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874 (2021)
5. Huang, Y., Zeng, W., Zeng, X., Zhu, Q., He, J.: From Accuracy to Robustness: A Study of Rule- and Model-based Verifiers in Mathematical Reasoning. arXiv:2505.22203 (2025)
6. Kim, J., Luo, X., Kim, M., Lee, S., Kim, D., Jeon, J., Li, D., Yang, Y.: Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs? arXiv:2603.24472 (2026)
7. Qiao, X., Du, X., Liu, W., Zhang, J., Mai, P., Zhang, M., Pang, Y.: When Sample Selection Bias Precipitates Model Collapse. arXiv:2606.13732 (2026)
8. Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., Gal, Y.: AI models collapse when trained on recursively generated data. Nature (2024). https://doi.org/10.1038/s41586-024-07566-y
9. Singh, A., Co-Reyes, J.D., Agarwal, R., et al.: Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models. arXiv:2312.06585 (2023)
10. Gulcehre, C., Le Paine, T., Srinivasan, S., et al.: Reinforced Self-Training (ReST) for Language Modeling. arXiv:2308.08998 (2023)
11. Strozzi, I.L.: Teacher-Free Self-Training Amplifies but Does Not Compound: A Pass@k Crossover on a Free-Verifier Domain. arXiv:2606.07856 (2026)
12. Xu, C., Yan, N., Chen, L., Kechadi, M.-T.: Phantom Gains: Auditing Self-Improvement Against a Measured Null. arXiv:2608.20290 (2026)
13. Xu, Z., Li, Y., Jiang, F., Ramasubramanian, B., Niu, L., Lin, B., Poovendran, R.: TinyV: Reducing False Negatives in Verification Improves RL for LLM Reasoning. arXiv:2505.14625 (2025)
14. Yuan, Z., Yuan, H., Li, C., Dong, G., Lu, K., Tan, C., Zhou, C., Zhou, J.: Scaling Relationship on Learning Mathematical Reasoning with Large Language Models. arXiv:2308.01825 (2023)

15. Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Yue, Y., Song, S., Huang, G.: Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837 (2025)
16. Zelikman, E., Wu, Y., Mu, J., Goodman, N.D.: STaR: Bootstrapping Reasoning With Reasoning. arXiv:2203.14465 (2022)